STACKLY SALEM

ONLINE EXAM PREPARATION & LEARNING PLATFORM

Technical Documentation

---

1. Project Overview

Project Name: Stackly — Online Exam Preparation & Learning Platform

Project Type: Frontend Web Application / Educational Platform

Architecture: Static Multi-Page HTML Shell + Vanilla JavaScript SPA-style Routing

Primary Focus: Online examination preparation, mock testing, courses, academic resources, student analytics, and examination administration

Branding: Stackly Salem

Headquarters Reference: Salem, Tamil Nadu, India

The project is a browser-based examination preparation and learning platform designed to support multiple categories of competitive, academic, professional, technical, and specialized examinations.

The platform provides:

- Exam discovery and preparation
- Course catalog
- Interactive mock examinations
- Question answering and scoring
- Student dashboard
- Learning progress tracking
- Error/question analysis
- Administrative examination monitoring
- Proctoring interface simulation
- Course syllabus inspection
- Academic services
- Blog and study resources
- Study cheat sheets
- Authentication interfaces
- Campus/contact information
- Responsive web design

The implementation is primarily client-side and uses JavaScript-driven rendering rather than a conventional server-side application.

---

2. Core Objectives

The application is designed around several primary objectives:

1. Provide a centralized exam-preparation platform.
2. Support multiple examination categories.
3. Offer realistic mock-test simulations.
4. Allow students to monitor academic progress.
5. Provide structured courses and syllabi.
6. Provide administrators with examination-operation tools.
7. Simulate proctored examination workflows.
8. Present academic articles and preparation resources.
9. Provide a modern, responsive learning interface.
10. Establish Stackly Salem as an examination and learning technology platform.

---

3. Technology Stack

Frontend

- HTML5
- CSS3
- Vanilla JavaScript
- JavaScript ES6+ patterns
- Tailwind CSS
- CSS custom properties
- Responsive layouts
- DOM-based rendering

External Libraries / CDNs

Tailwind CSS

The project loads Tailwind CSS through its CDN:

```
https://cdn.tailwindcss.com
```

Tailwind is configured directly inside the HTML documents.

Lucide Icons

The application uses Lucide for interface icons:

```
https://unpkg.com/lucide@latest
```

Google Fonts

The visual system uses:

- Outfit
- Plus Jakarta Sans

These fonts are loaded from Google Fonts.

Image Assets

The project contains numerous WebP image assets under:

```
assets/images/
```

The project also contains:

```
assets/logo.svg
```

Deployment Automation

GitHub Actions configuration is included at:

```
.github/workflows/static.yml
```

This indicates that the application is structured for static hosting/deployment.

---

4. Project Structure

The primary application structure is:

```
Online Exam Preparation/
│
├── index.html
├── about.html
├── admin-dashboard.html
├── blog.html
├── contact.html
├── courses.html
├── not-found.html
├── services.html
```

```
├── student-dashboard.html
│
├── assets/
│   ├── images/
│   │   └── *.webp
│   └── logo.svg
│
├── css/
│   └── styles.css
│
├── js/
│   ├── app.js
│   ├── data.js
│   │
│   ├── components/
│   │   ├── auth-modal.js
│   │   ├── exam-modal.js
│   │   ├── footer.js
│   │   ├── navbar.js
│   │   └── syllabus-modal.js
│   │
│   └── pages/
│       ├── about.js
│       ├── admin-dashboard.js
│       ├── blog.js
│       ├── contact.js
│       ├── courses.js
│       ├── home.js
│       ├── not-found.js
│       ├── services.js
│       └── student-dashboard.js
│
└── .github/
    └── workflows/
        └── static.yml
```

The archive also contains Git metadata under `.git`.

---

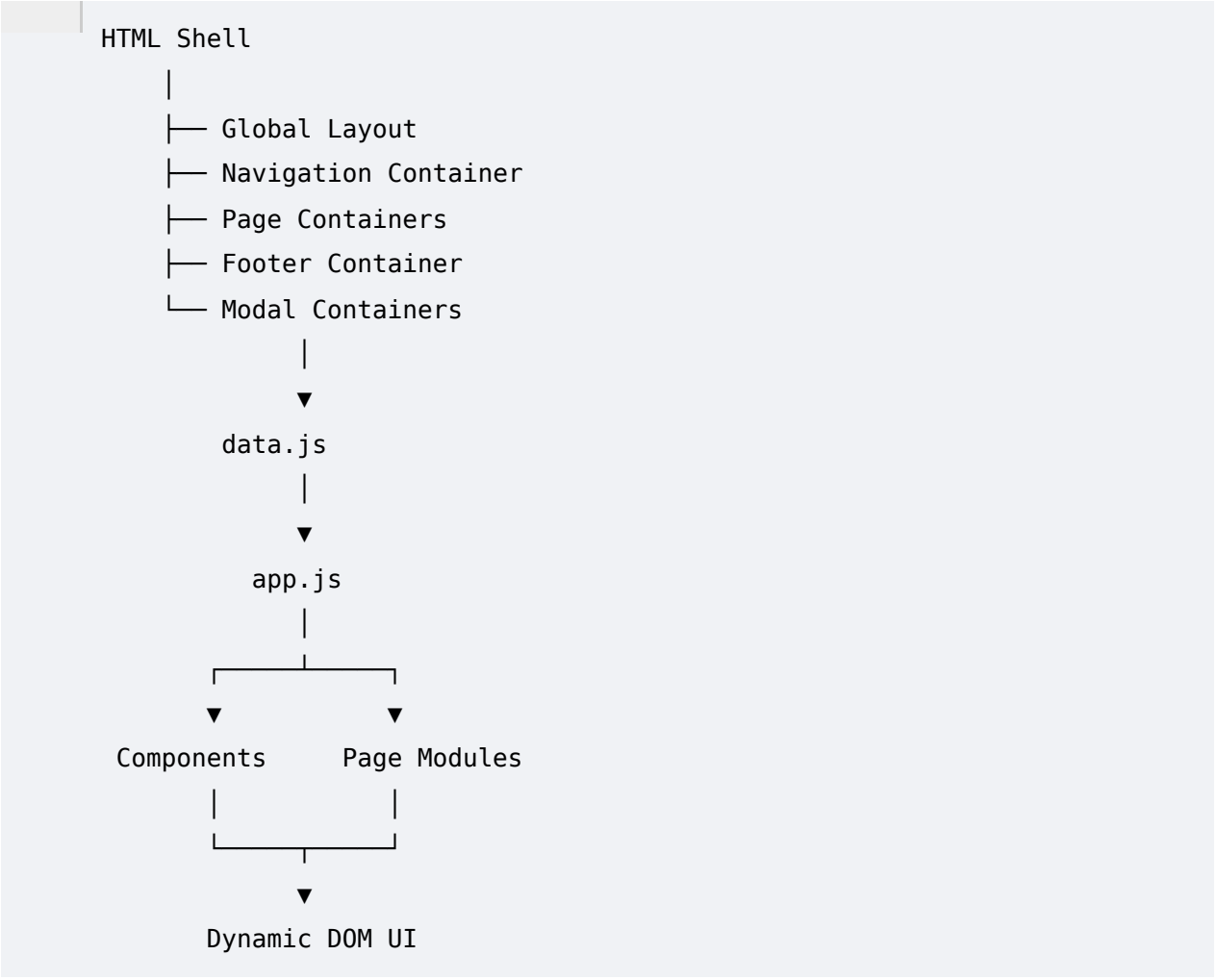
5. Application Architecture

The application follows a hybrid architecture.

The project contains multiple HTML entry files, but each HTML file contains a common application shell.

The page-specific content is largely generated dynamically by JavaScript.

The architecture can therefore be described as:



---

6. HTML Application Shell

Each main HTML file establishes the application framework.

Important shared containers include:

```
#main-navbar
#main-footer
#auth-modal-backdrop
#exam-modal-backdrop
#syllabus-modal-backdrop
#toast-container
```

Page containers use the pattern:

```
#page-home
#page-about
#page-services
#page-courses
#page-blog
#page-contact
#page-student-dashboard
#page-admin-dashboard
#page-not-found
```

Only the relevant route is marked active.

---

7. Main Application State

The main application state is defined in:

```
js/app.js
```

The primary state object is:

```
const AppState = {
  currentRoute: null,
  previousRoute: 'home',
  user: null,
  isAuthOpen: false,
  isExamOpen: false,
  isSyllabusOpen: false,
  activeCourse: null,
  pendingRoute: null,
  pendingExamTitle: null,
  authNotice: null,
  authInitialRole: 'student',
  toastMessage: null,
  theme: 'royal',
  isAnnualPricing: true,
  isNavigating: false,
};
```

This object coordinates:

- Current route
- Previous route

- Authenticated user
- Modal state
- Active course
- Exam state
- Authentication state
- Theme
- Pricing display mode
- Navigation state

---

8. Event Bus

The application implements a lightweight event bus.

The event system provides:

```
Events.on()
Events.off()
Events.emit()
```

This allows modules to communicate without requiring a large framework.

For example, route changes can emit:

```
route-changed
```

This approach provides loose coupling between application modules.

---

9. Client-Side Router

Routing is handled through JavaScript.

The main routing function is:

```
navigate(route, options)
```

The router:

1. Checks authentication requirements.
2. Locates the target page container.
3. Updates `AppState`.
4. Updates the URL hash.
5. Removes the active state from other pages.

6. Activates the target page.
7. Calls the appropriate page rendering function.
8. Updates navigation state.
9. Closes menus/dropdowns.
10. Scrolls to the top.
11. Refreshes scroll-reveal animations.
12. Emits a route-change event.

Routes include:

```
home
about
services
courses
blog
contact
student-dashboard
admin-dashboard
not-found
```

---

10. Route Labels

The project maintains metadata for routes.

Examples include:

```
Home Dashboard
Institutional Profile
Academic Infrastructure
Curriculum Vault
Research Publications
Campus Navigator
Candidate Portal
Administration Suite
404 Diagnostic
```

These labels are also used by the application's preloader.

---

11. Preloader System

The application includes a full-screen loading transition.

The preloader contains:

- Animated spinner
- Stackly branding
- Progress percentage
- Progress bar
- Loading status
- Route-specific labels
- Skip-loading control

The loader progresses toward 100% using a timed JavaScript interval.

The user can bypass the loader through:

```
Skip loading
```

---

12. Scroll Progress

The application contains a fixed progress indicator at the top of the browser window.

The JavaScript calculates:

```
current scroll position
-----------------------
total scrollable height
```

and transforms the progress bar accordingly.

This gives the user an indication of reading progress.

---

13. Scroll Reveal Animation

The application uses `IntersectionObserver` to detect when elements enter the viewport.

Elements with classes such as:

```
.reveal
.stagger-grid
```

can receive a visible state.

The system also includes fallback behavior for browsers without `IntersectionObserver`.

---

14. Data Architecture

The primary mock/content database is:

```
js/data.js
```

The application defines numerous global data collections.

Major collections include:

```
EXAM_CATEGORIES
COURSES
FACULTY_MEMBERS
TESTIMONIALS
MOCK_QUESTIONS
PRICING_PLANS
PREP_CENTERS
SALEM_COMPANY_INFO
FAQS
HIGH_YIELD_RESOURCES
STUDENT_DASHBOARD_DATA
ADMIN_DASHBOARD_DATA
COLOR_THEMES
SERVICES_LIST
BLOG_POSTS
STUDY_CHEATSHEETS
DEFAULT_REGISTERED_USERS
DEFAULT_NOTIFICATIONS
```

This makes `data.js` the central content and mock-data layer.

---

15. Exam Categories

The application supports a broad set of examinations.

Examples include:

- GRE
- GMAT
- MCAT
- USMLE
- SAT
- GATE

- IELTS
- LSAT
- TNPSC
- Indian Railways RRB
- Banking / IBPS / SBI
- AI & Machine Learning
- Programming & Software Engineering
- History & General Knowledge
- Kids Olympiad
- Senior Citizens / Lifelong Learning

Each examination category can contain:

- ID
- Name
- Exam code
- Category
- Description
- Score improvement indicator
- Student count
- Duration
- Visual badge
- Icon
- Popular subjects

---

16. GRE Category

The GRE category is represented as a graduate-level examination preparation program.

Major subjects include:

- Quantitative Reasoning
- Verbal Reasoning
- Analytical Writing

The application presents diagnostic and preparation-oriented statistics around expected score improvement.

---

17. GMAT Category

The GMAT Focus Edition preparation category focuses on:

- Data Insights
- Quantitative reasoning
- Verbal reasoning
- Critical reasoning
- Scoring strategy

The course catalog also includes a dedicated GMAT preparation course.

---

18. Medical Examination Categories

Medical preparation includes:

- MCAT
- USMLE Step 1
- USMLE Step 2 CK

The data model supports specialized medical preparation topics such as:

- Biology
- Biochemistry
- Chemistry
- Physics
- Pathology
- Pharmacology
- Microbiology
- Cardiovascular systems
- Clinical vignettes

---

19. Indian Competitive Examinations

The project has extensive Indian exam-oriented content.

Important examples include:

TNPSC

Supports:

- Group 1
- Group 2
- Group 4

- General Studies
- Aptitude
- Mental Ability
- Tamil
- Thirukkural
- Current Affairs
- Tamil Nadu administration

Indian Railways

Supports preparation for:

- RRB NTPC
- ALP
- Group D
- Junior Engineer

Banking

Supports:

- IBPS
- SBI
- RBI Grade B
- Specialist Officer

---

20. Technical / Professional Preparation

The platform also contains technology-oriented learning.

The AI/ML preparation category covers subjects such as:

- Deep Learning
- Large Language Models
- PyTorch
- Transformers
- Computer Vision
- MLOps

Programming preparation includes:

- Python
- Data Structures

- Algorithms
- JavaScript
- TypeScript
- SQL
- System Design
- Cloud / DevOps

---

21. Specialized Learner Categories

The project also attempts to support broader demographics.

These include:

Kids

- Mathematics
- Science
- Logic
- Language
- Olympiad-style questions

Senior Citizens

- Memory wellness
- Brain fitness
- Digital banking
- Cybersecurity literacy
- Heritage studies
- Lifelong learning

---

22. Course Catalog

Course information is stored in:

```
COURSES
```

Each course contains information such as:

- Course ID
- Title
- Examination
- Category

- Level
- Instructor
- Rating
- Review count
- Enrollment count
- Duration
- Video hours
- Practice question count
- Full mock count
- Original price
- Sale price
- Badge
- Highlights
- Syllabus

---

23. Course Instructor Model

Course instructors contain information including:

```
name
role
avatar
rating
```

This supports faculty-oriented course presentation.

For example, the GRE course contains an instructor profile with:

- Name
- Academic credentials
- Professional role
- Avatar
- Rating

---

24. Course Pricing

Courses contain both:

```
originalPrice
salePrice
```

This allows the interface to show promotional pricing.

The global application also maintains:

```
isAnnualPricing
```

which supports switching between pricing periods where applicable.

---

25. Course Syllabus

Each course can contain multiple modules.

Each module contains:

```
moduleTitle
lessons
quizzesCount
```

The application therefore supports a hierarchical structure:

```
Course
  ├── Module
  │     ├── Lesson
  │     ├── Lesson
  │     └── Quiz count
  │
  └── Module
        ├── Lesson
        └── Quiz count
```

---

26. Syllabus Modal

The project contains:

```
js/components/syllabus-modal.js
```

This component displays course curriculum details without requiring a separate page.

The modal architecture supports:

- Course title
- Modules

- Lessons
- Quiz counts
- Course metadata
- Interactive inspection

---

27. Mock Examination Engine

The main examination simulator is implemented in:

```
js/components/exam-modal.js
```

The primary object is:

```
ExamEngine
```

It manages the complete mock-test interaction.

---

28. Exam Engine State

Important exam state variables include:

```
selectedDomain
currentIdx
selectedAnswers
flagged
secondsRemaining
isSubmitted
showConfirmSubmit
timerInterval
```

This allows the engine to manage a complete question-by-question examination session.

---

29. Examination Domains

The mock-test engine supports domain filtering for:

```
All
TNPSC
Railways
Banking
AI / Machine Learning
Programming
```

```
History / GK
Kids
Senior
Global
```

The Global category covers examination families such as:

- GRE
- GMAT
- SAT

---

30. Mock Question Model

The examination engine consumes:

```
MOCK_QUESTIONS
```

Questions can be filtered by:

- Category
- Exam
- Question ID

The engine determines the correct answer from the stored question data.

---

31. Examination Timer

The mock examination begins with:

```
600 seconds
```

which corresponds to:

```
10 minutes
```

The timer is maintained using JavaScript's `setInterval()`.

The displayed format is:

```
MM:SS
```

When less than two minutes remain, the timer changes to an alert-oriented visual state.

---

32. Answer Selection

Students can select an answer option.

The answer is stored against the question ID:

```
selectedAnswers[questionId]
```

The interface is then re-rendered to reflect the selected answer.

---

33. Question Flagging

Students can flag questions for later review.

The state is maintained through:

```
flagged
```

This allows the interface to distinguish questions requiring additional attention.

---

34. Exam Scoring

When the test is submitted, the engine compares:

```
selected answer
```

against:

```
correctAnswer
```

for each question.

It calculates:

- Correct answers
- Accuracy percentage
- Projected percentile

The interface therefore provides an immediate diagnostic result.

---

35. Projected Percentile

The mock engine contains threshold-based percentile estimation.

The current implementation maps accuracy into approximate percentile ranges.

This should be considered a **simulation**, not an official examination scoring model.

A production platform should calculate percentiles using validated examination-specific scoring data.

---

36. Student Dashboard

The student portal is located at:

```
student-dashboard.html
```

and implemented primarily through:

```
js/pages/student-dashboard.js
```

The dashboard is designed as an academic operations center.

---

37. Student Dashboard Features

The student dashboard supports areas such as:

- Academic overview
- Course progress
- Resume lesson
- Error/question analysis
- Study progress
- Test history
- Performance information
- Learning state

The dashboard uses:

```
STUDENT_DASHBOARD_DATA
```

for its initial mock state.

---

38. Resume Lesson Player

The student dashboard contains an interactive lesson player.

The `resumeLesson()` function creates a modal dynamically.

The simulated player includes:

- Course information
- Instructor

- Lecture title
- Video frame
- Play button
- HD stream label
- Playback progress
- Lecture controls

The video experience is currently a frontend simulation rather than a real streaming service.

---

39. Cognitive Error Matrix

The student dashboard includes an error-analysis concept.

Questions can be marked:

```
resolved
```

or unresolved.

The state is maintained in the dashboard's:

```
errorItems
```

collection.

This supports a study workflow where students can identify and resolve recurring mistakes.

---

40. Authentication

Authentication is implemented primarily on the frontend.

The component is:

```
js/components/auth-modal.js
```

The authentication interface supports:

- Sign in
- Account creation
- Student role
- Administrator role
- Authentication notices

- Modal-based login

---

41. Authentication Roles

The application distinguishes between:

```
student
admin
```

Student users access the academic portal.

Administrators access the operations console.

The router performs role checks before allowing access to the admin dashboard.

---

42. Authentication Guard

The router protects:

```
student-dashboard
admin-dashboard
```

For students, an unauthenticated visitor is directed toward login.

For administrators, the router checks:

1. User authentication
2. User role

A non-admin user cannot directly access the administrator route through the normal navigation guard.

---

43. Important Security Limitation

The current authentication implementation is a **frontend demonstration**.

The application does not contain a production backend authentication service.

Therefore:

- Credentials are not protected by a server.
- Roles are controlled client-side.
- Authorization cannot be trusted in production.
- Client-side JavaScript can be modified.
- Sensitive user information should not be stored in this architecture.

A production implementation must move authentication and authorization to a secure backend.

---

44. Administrator Dashboard

The administrator dashboard is available at:

```
admin-dashboard.html
```

Its main controller is:

```
js/pages/admin-dashboard.js
```

The dashboard is presented as a:

```
Salem HQ Admin Operations Console
```

---

45. Administrative Operations

The administrator interface supports operational concepts such as:

- Live examinations
- Proctor monitoring
- Examination filtering
- Question-bank summaries
- Search
- Session controls
- Session flagging
- Live-feed inspection
- Question management

---

46. Live Examination Sessions

The administrator dashboard consumes:

```
ADMIN_DASHBOARD_DATA.liveExams
```

The information can represent:

- Session ID
- Student

- Examination
- Proctoring status
- Examination state

---

47. Proctoring Interface

The administrator dashboard contains a simulated live proctoring interface.

The `inspectLiveFeed()` function creates a modal containing:

- Primary webcam view
- Secondary monitoring area
- Student identity
- Examination name
- Session ID
- Proctor status
- Facial tracking visualization
- Gaze indicator

The interface visually simulates AI-assisted examination monitoring.

---

48. Proctoring Status

The interface contains simulated indicators such as:

```
FACE CONFIRMED
GAZE: ON-SCREEN
LIVE PROCTOR FEED
```

These values are presentation-layer simulations.

They do not represent an actual computer-vision or biometric system.

---

49. Session Controls

The administrator can perform simulated operational actions such as:

```
Pause session
Flag session
Inspect live feed
```

Flagging changes the local session state to indicate that a proctor is investigating.

---

50. Question Bank Management

The admin dashboard maintains:

```
questionList
```

which is initialized from:

```
questionBankSummary
```

The interface can be used to visualize examination question-bank information and provide a management-oriented workflow.

---

51. Services Page

The services module is implemented through:

```
js/pages/services.js
```

The source data is:

```
SERVICES_LIST
```

The services section represents Stackly's broader academic and examination infrastructure.

The content includes services associated with:

- Mock examinations
- Proctored testing
- Academic preparation
- Testing facilities
- Learning support
- Examination infrastructure

---

52. Services Calculations

The services module includes client-side calculations involving values such as:

```
weeklyHours
mockTests
estimatedTuition
```

This suggests that portions of the services experience are interactive rather than purely informational.

---

53. Blog Module

The blog functionality is implemented through:

```
js/pages/blog.js
```

Data is stored in:

```
BLOG_POSTS
```

The platform presents the blog as an examination research and preparation publication area.

---

54. Blog Features

The blog module supports:

- Featured posts
- Category filtering
- Search
- Article cards
- Exam-specific content
- High-yield preparation information

The module also uses:

```
STUDY_CHEATSHEETS
```

for downloadable/reference-oriented resources.

---

55. Study Cheat Sheets

The data model supports cheat sheets with fields such as:

```
id
title
exam
fileSize
```

```
downloads
badge
```

Examples include high-yield quantitative and examination preparation resources.

These are currently content records within the frontend data layer.

---

56. About Page

The About page is implemented through:

```
js/pages/about.js
```

It presents institutional information around Stackly and the Salem headquarters.

The page also uses faculty data.

---

57. Faculty Directory

Faculty data is stored in:

```
FACULTY_MEMBERS
```

Faculty records include information such as:

- Name
- Role
- Credentials
- Institution
- Specialization
- Profile image

This provides the foundation for an academic faculty presentation system.

---

58. Testimonials

Student success stories are stored in:

```
TESTIMONIALS
```

Records include fields such as:

- Student name
- Examination

- Score
- Previous score
- Percentile
- Target school
- Program

This supports performance-oriented social proof on the platform.

---

59. Pricing Plans

Pricing information is stored in:

```
PRICING_PLANS
```

Plans contain:

- Plan ID
- Plan name
- Price
- Billing period
- Description
- Feature list

The interface can therefore support multiple subscription tiers.

---

60. Preparation Centers

Physical preparation/testing locations are represented using:

```
PREP_CENTERS
```

The data contains:

- City
- Country
- Address
- Phone
- Center information

The primary branded location is the Salem headquarters.

---

61. Contact Page

The contact module is implemented through:

```
js/pages/contact.js
```

It consumes:

```
PREP_CENTERS
SALEM_COMPANY_INFO
```

The interface provides location-oriented information for students and visitors.

---

62. FAQ System

The project contains:

```
FAQS
```

within its central data layer.

This provides a foundation for frequently asked examination-preparation and platform questions.

---

63. High-Yield Resources

The data layer includes:

```
HIGH_YIELD_RESOURCES
```

These resources can be used to present supplemental academic content beyond the main course catalog.

---

64. Navigation Component

The navigation component is:

```
js/components/navbar.js
```

It is shared across application pages.

The navbar handles:

- Site navigation
- Current route indication
- Responsive menu behavior

- Authentication actions
- Dashboard navigation

---

65. Footer Component

The shared footer is implemented through:

```
js/components/footer.js
```

This avoids duplicating footer markup across the application's individual HTML files.

---

66. Authentication Modal Component

The authentication UI is encapsulated in:

```
js/components/auth-modal.js
```

The component dynamically generates the login/register interface.

It supports role-specific authentication messaging and default account context.

---

67. Exam Modal Component

The exam system is implemented as a reusable modal instead of requiring a dedicated exam page.

Advantages include:

- Fast access from any page
- Reusable examination engine
- No additional navigation
- Centralized exam state
- Easy diagnostic-launch workflow

---

68. Global Toast System

The HTML shell contains:

```
#toast-container
```

JavaScript modules call:

```
showToast()
```

to communicate actions to the user.

This is used for feedback such as:

- Authentication status
- Session actions
- Question updates
- Student dashboard actions
- Administrative operations

---

69. Theme System

The CSS defines theme variables using custom properties.

The default theme is:

```
royal
```

Additional theme definitions include:

```
royal
emerald
sunset
```

The theme system changes:

- Background
- Primary color
- Secondary color
- Accent
- Borders
- Badges
- Hero gradients
- Glow effects
- Navigation colors

---

70. Visual Design System

The platform uses a modern educational SaaS visual language.

Major characteristics include:

- Indigo primary color

- Cyan secondary accents
- Emerald success states
- Amber highlights
- White cards
- Slate backgrounds
- Rounded corners
- Soft shadows
- Gradient backgrounds
- Glass-like overlays
- Animated UI elements

---

71. Typography

The main typography system uses:

Plus Jakarta Sans

Used primarily for:

- Body text
- Navigation
- Interface labels
- General UI

Outfit

Used primarily for:

- Headings
- Display text
- Major visual labels

This provides a combination of readable body typography and stronger display typography.

---

72. Responsive Design

The platform uses responsive Tailwind utility classes and custom CSS.

Layouts adapt for:

- Mobile
- Tablet

- Desktop
- Large desktop screens

Responsive behavior includes:

- Stacked layouts
- Responsive grids
- Mobile navigation
- Flexible cards
- Modal resizing
- Responsive typography
- Adaptive dashboard layouts

---

73. Image Asset System

The project includes approximately 29 explicitly listed WebP image assets in the main asset directory.

Images are used for:

- Faculty
- Course imagery
- Student/proctoring visuals
- Promotional sections
- Content cards
- General platform branding

The project also includes an SVG logo.

---

74. Error Page

The application includes:

```
not-found.html
```

and:

```
js/pages/not-found.js
```

The page is branded as a resource recovery/diagnostic experience rather than using a generic browser-style 404 screen.

---

75. Main Page Modules

The application uses page-specific JavaScript controllers.

The main modules are:

```
HomeModule
AboutModule
ServicesModule
CoursesModule
BlogModule
ContactModule
StudentDashboardModule
AdminDashboardModule
```

This provides a modular frontend structure without requiring a large JavaScript framework.

---

76. Home Module

The home page is implemented in:

```
js/pages/home.js
```

It acts as the primary marketing and platform entry point.

The module contains functionality related to:

- Exam discovery
- Performance/score estimation
- Learning statistics
- Preparation messaging
- Platform highlights
- Calls to action

---

77. Courses Module

The courses page is implemented in:

```
js/pages/courses.js
```

It supports:

- Course rendering

- Search
- Examination filtering
- Level filtering
- Course cards
- Syllabus inspection
- Course-related actions

---

78. Blog Module

The blog module provides:

```
category filtering
search
featured content
article presentation
study resources
```

It is designed to act as an academic content layer around the core exam engine.

---

79. Contact Module

The contact module supports:

- Preparation center selection
- Center information
- Location details
- Institutional contact information

---

80. Static Data vs Dynamic Data

A major architectural characteristic is that most data is currently static.

The following are frontend-defined:

```
Users
Courses
Questions
Faculty
Testimonials
Pricing
Services
```

```
Blogs
Dashboard data
Admin data
Notifications
```

This is suitable for:

- Prototype demonstrations
- UI development
- Frontend testing
- Portfolio presentations
- Static deployments

It is not sufficient by itself for a production examination platform.

---

81. Current Backend Status

No conventional backend application was identified in the project structure.

There is no clear implementation of:

- REST API server
- Node/Express backend
- Django backend
- Laravel backend
- Spring backend
- PostgreSQL database
- MySQL database
- MongoDB database

The current implementation is therefore primarily frontend/client-side.

---

82. Database Status

There is no production database layer in the supplied project.

The application relies on JavaScript data objects for its content and simulated state.

A production deployment would require persistent server-side storage.

---

83. Production Data Architecture Recommendation

A production implementation could use:

```
Frontend
   │
   ▼
API Gateway / Backend
   │
   ├── Authentication Service
   ├── Exam Service
   ├── Question Bank Service
   ├── Course Service
   ├── Student Analytics
   ├── Proctoring Service
   └── Administration Service
            │
            ▼
        Database
```

---

84. Recommended Production Database Entities

A production database could contain:

```
Users
Roles
Permissions
StudentProfiles
AdminProfiles
Exams
ExamCategories
Questions
QuestionOptions
QuestionTags
QuestionAttempts
ExamAttempts
ExamResults
Courses
CourseModules
Lessons
Quizzes
Enrollments
Faculty
Testimonials
PricingPlans
```

```
Subscriptions
Services
BlogPosts
StudyResources
PreparationCenters
Notifications
ProctorSessions
ProctorEvents
AuditLogs
```

---

85. Authentication Improvements

For production, authentication should be implemented server-side.

Recommended controls include:

- Secure password hashing
- HTTPS
- Secure cookies
- Session management
- Server-side role validation
- Multi-factor authentication for administrators
- Password reset
- Account verification
- Rate limiting
- Login auditing

---

86. Authorization Improvements

The current frontend role check should be replaced or supplemented with backend RBAC.

Recommended roles include:

```
Student
Instructor
Proctor
Admin
Super Admin
Content Manager
Support Agent
```

Every privileged API operation should validate the user's role server-side.

---

87. Examination Security

A production examination system should additionally protect:

- Question delivery
- Correct answers
- Exam timing
- Attempt state
- Score calculation
- Exam submission
- Result generation

Correct answers should never be shipped to the browser before they are required.

Otherwise, technically knowledgeable users could inspect the JavaScript/data and discover answer keys.

---

88. Proctoring Security

The current proctoring interface is a visual simulation.

A real implementation would require:

- Browser camera permissions
- Microphone permissions
- Secure media streaming
- Identity verification
- Face matching where legally appropriate
- Event detection
- Human proctor review
- Privacy controls
- Data retention policy
- Consent management

Any biometric or surveillance functionality should also be implemented with appropriate legal and privacy safeguards.

---

89. Payment Architecture

The course/pricing layer currently represents pricing information in frontend data.

For real subscriptions, a payment service should be integrated through a secure backend.

The frontend should never directly trust:

```
price
payment status
subscription status
```

as authoritative values.

---

90. Analytics Architecture

The platform's academic analytics could be expanded into server-side metrics such as:

```
Attempts per student
Accuracy
Question difficulty
Time per question
Topic accuracy
Course completion
Mock-test scores
Percentile history
Weak subjects
Improvement rate
Study hours
Retention
```

These could form the basis of a real learning analytics system.

---

91. Performance Recommendations

For production deployment:

1. Build Tailwind CSS rather than relying on the CDN.
2. Minify JavaScript and CSS.
3. Compress and resize images.
4. Lazy-load noncritical images.
5. Use a CDN for static assets.
6. Cache immutable assets.

7. Split large JavaScript modules where useful.
8. Avoid unnecessary full DOM re-rendering.
9. Preload critical fonts carefully.
10. Optimize the initial page payload.

---

92. Maintainability

The project has several maintainability strengths.

Strengths

- Modular JavaScript
- Centralized data
- Shared components
- Separate page modules
- Reusable modal architecture
- Central application state
- Consistent CSS variables
- Clear route structure

Areas for improvement

- Global variables are heavily used.
- There is no module bundler.
- Data and UI concerns are closely connected.
- Large page modules can become difficult to maintain.
- HTML is duplicated as page shells.
- Client-side data is not normalized.
- Production type checking is absent.

---

93. JavaScript Architecture Improvement

For a larger production application, the current global-script architecture could eventually be migrated toward:

```
ES Modules
TypeScript
Component framework
State management layer
API service layer
```

For example:

```
src/
├── components/
├── pages/
├── services/
├── store/
├── models/
├── utils/
└── api/
```

---

94. Accessibility

The project contains several accessibility-conscious elements such as:

- Semantic buttons
- Labels
- Alternative text for images
- Modal close controls
- Responsive layouts

However, a formal accessibility audit should still be performed.

Recommended checks include:

- Keyboard navigation
- Focus trapping
- Focus restoration
- Screen-reader announcements
- Color contrast
- Form labeling
- Error messaging
- Reduced-motion support
- Accessible modal semantics

---

95. Browser Compatibility

The application is intended for modern browsers supporting:

- ES6 JavaScript
- CSS custom properties

- IntersectionObserver
- Modern DOM APIs
- Responsive CSS

Recommended testing targets:

```
Google Chrome
Microsoft Edge
Mozilla Firefox
Safari
Android Chrome
iOS Safari
```

---

96. Testing Strategy

A complete production test plan should include:

Functional Testing

- Navigation
- Login
- Registration
- Course filtering
- Search
- Exam start
- Answer selection
- Flagging
- Submission
- Scoring
- Dashboard access
- Admin access

UI Testing

- Responsive layouts
- Modals
- Dropdowns
- Navigation
- Loading states
- Toasts

Security Testing

- Unauthorized route access
- Role escalation
- Input manipulation
- Session tampering
- Question-answer exposure

---

97. Exam Engine Testing

Specific mock-test tests should verify:

1. Timer starts correctly.
2. Timer stops on submission.
3. Answers persist while navigating.
4. Flagging works.
5. Domain filters return appropriate questions.
6. Empty question collections are handled.
7. Score calculation is correct.
8. Submission state cannot accidentally reset.
9. Timeout submits correctly.
10. Multiple attempts do not leak state.

---

98. Administrative Testing

The administrator interface should be tested for:

- Admin-only access
- Session filtering
- Session searching
- Session pausing
- Session flagging
- Live-feed modal behavior
- Question-bank management
- State refresh

---

99. Deployment

The inclusion of:

```
.github/workflows/static.yml
```

indicates that the project is designed to work with static hosting and automated GitHub-based deployment.

Possible deployment environments include static hosting services that support HTML/CSS/JavaScript applications.

---

100. Local Development

The project can be served using a local static server.

A typical development command is:

```
npx serve -p 3000 .
```

The site can then be accessed through the local development server.

Using a local server is preferable to opening HTML files directly because browser security restrictions can affect local asset and JavaScript behavior.

---

101. Version Control

The project includes a Git repository:

```
.git/
```

This indicates that the source has been managed through Git version control.

The deployment workflow is also stored alongside the source code.

---

102. Project Strengths

The project demonstrates several strong frontend engineering practices.

1. Modular architecture

Page logic is separated into individual JavaScript modules.

2. Reusable components

Navbar, footer, authentication, exam, and syllabus interfaces are separated into reusable components.

3. Rich mock data

The data layer provides realistic structures for exams, courses, faculty, services, questions, dashboards, and resources.

4. Interactive examination engine

The mock exam includes timer, answer selection, flagging, filtering, and scoring.

5. Student and administrator experiences

The application demonstrates both sides of an examination platform.

6. Responsive interface

The UI is designed for multiple screen sizes.

7. Strong visual identity

The Stackly Salem branding is consistent across the platform.

---

103. Technical Limitations

The major limitations are architectural rather than visual.

No production backend

The platform currently operates primarily in the browser.

No production database

Data is defined in JavaScript rather than persistent server storage.

Client-side authentication

Authentication and roles cannot be considered secure.

Simulated proctoring

The proctoring interface is visual rather than a real proctoring service.

Simulated examination scoring

The percentile calculation is illustrative.

No real payment processing

Pricing exists, but a production billing system is not implemented.

No real-time exam infrastructure

Live examination sessions are represented through frontend data.

---

104. Recommended Production Roadmap

Phase 1 — Backend Foundation

Implement:

- REST/GraphQL API
- Authentication
- User management
- Database
- Course APIs
- Exam APIs
- Question APIs

Phase 2 — Examination Platform

Implement:

- Secure exam sessions
- Persistent attempts
- Server-side scoring
- Exam scheduling
- Result storage
- Question randomization
- Time enforcement

Phase 3 — Learning Management

Implement:

- Course enrollment
- Lesson tracking
- Video delivery
- Quiz system
- Assignments
- Certificates

Phase 4 — Analytics

Implement:

- Student performance dashboards
- Topic analysis
- Score trends
- Percentile trends
- Personalized recommendations

Phase 5 — Administration

Implement:

- Admin RBAC
- Question management
- Exam creation
- Student management
- Proctor management
- Audit logs

Phase 6 — Proctoring

Implement secure and privacy-compliant:

- Camera integration
- Microphone integration
- Proctor sessions
- Event monitoring
- Human review

---

105. Recommended API Structure

A production API could be organized approximately as:

```
/api/auth
/api/users
/api/students
/api/admin
/api/exams
/api/questions
/api/attempts
/api/results
/api/courses
/api/lessons
/api/enrollments
/api/resources
/api/blog
/api/proctoring
/api/notifications
/api/payments
```

---

106. Suggested Frontend Architecture

A future production frontend could evolve toward:

```
src/
│
├── components/
│   ├── Navbar
│   ├── Footer
│   ├── Modal
│   ├── ExamEngine
│   └── CourseCard
│
├── pages/
│   ├── Home
│   ├── Courses
│   ├── Exam
│   ├── StudentDashboard
│   └── AdminDashboard
│
├── services/
│   ├── auth
│   ├── exams
│   ├── courses
│   └── analytics
│
├── store/
│   └── applicationState
│
├── models/
│   ├── User
│   ├── Exam
│   ├── Course
│   └── Question
│
└── utils/
```

---

107. Suggested Security Model

The production security model should follow:

```
Browser
   │
   │ HTTPS
   ▼
API
   │
   ├── Authentication
   ├── Authorization
   ├── Validation
   ├── Rate Limiting
   └── Audit Logging
          │
          ▼
       Database
```

The browser should be treated as an untrusted client.

---

108. Recommended Data Ownership

Browser

Should manage:

- Temporary UI state
- Display preferences
- Non-sensitive cached information

Backend

Should manage:

- Accounts
- Roles
- Exam attempts
- Correct answers
- Scores
- Payments
- Enrollments
- Proctoring records
- Audit logs

---

109. Overall Technical Assessment

The supplied project is best classified as a **feature-rich frontend prototype / static educational platform**.

It goes beyond a simple landing page by implementing:

- Application routing
- Shared components
- Dynamic page rendering
- Course catalog
- Mock examination engine
- Student dashboard
- Administrative console
- Authentication interface
- Proctoring simulation
- Academic content
- Responsive UI
- Theming
- Loading states
- Interactive modals

The frontend foundation is sufficiently modular to serve as the starting point for a production learning-management and examination platform.

---

110. Final Conclusion

The Stackly Salem Online Exam Preparation project provides a comprehensive frontend implementation for a multi-exam educational platform.

Its strongest characteristics are its modular JavaScript architecture, extensive mock-data layer, interactive examination engine, course/syllabus system, student dashboard, administrative console, responsive design, and coherent visual identity.

The most important next step for production readiness is **backend integration**.

The current project should be treated as a frontend prototype or static demonstration until the following are implemented server-side:

- Secure authentication
- Role-based authorization

- Persistent database
- Secure examination sessions
- Server-side scoring
- Real course enrollment
- Payment processing
- Real-time administration
- Production-grade analytics
- Secure proctoring infrastructure

With those additions, the existing frontend can provide a strong foundation for a full-scale online examination and learning management platform.